Department of Computer Science and Engineering
University of Dhaka

Course Outline

CSE2101 - Data Structures and Algorithms

---

1. General Information

Course Title:	CSE2101 - Data Structures and Algorithms
Credit:	3
Semester:	January, 2023
Google Classroom Code:	mzqm6ng
Instructor:	Md. Tanvir Alam
Contact:	tanvir@cse.du.ac.bd

2. Course Contents

Introduction: Introduction to Data Structures, idea of abstract data type, preliminary idea of algorithm runtime complexity (Big Oh notation), preliminary idea of data structure space complexity. **LinkedList:** Singly/doubly/circular linked lists, basic operations on linked list (insertion, deletion and traverse), dynamic array and its application. **Stack and Queue:** Basic stack operations (push/pop/peek), stack-class implementation using Array and

linked list, in-fix to post-fix expressions conversion and evaluation, balancing parentheses using stack, basic queue operations(enqueue, dequeue), circular queue/ dequeue, queue-class implementation using array and linked list, application- Josephous problem, palindrome checker using stack and queue. **Recursion:** Basic idea of recursion (3 laws-base case, call itself, move towards base case by state change), tracing output of a recursive function, applications- merge sort, permutation, combination. **Sorting:** Insertion sort, selection sort, bubble sort, merge sort, quick sort (randomized quick sort), distribution sort (counting sort, radix sort, bucket sort), lower bounds for sorting, external sort. **Binary Tree:** Binary tree representation using array and pointers, traversal of Binary Tree (in-order, pre-order and post-order). **Binary Search Tree:** BST representation, basic operations on BST (creation, insertion, deletion, querying and traversing), application- searching, sets. **Searching:** Linear search, binary Search, application of Binary Search- finding element in a sorted array, finding n^{th} root of a real number, solving equations. **Heap:** Min-heap, max-heap, Fibonacci-heap, applications-priority queue, heap sort. **General Tree:** Implementation, application of general tree- file system**. Disjoint Set:** Union find, path compression. **Huffman Coding:** Implementation, application- Compression. **Graph:** Graph representation (adjacency matrix/adjacency list), basic operations on graph (node/edge insertion and deletion), traversing a graph: breadth-first search (BFS), depth-first search (DFS), graph-bicoloring. **Self-balancing Binary Search Tree:** AVL tree (rotation, insertion). **Set Operations:** Set representation using bitmask, set/clear bit, querying the status of a bit, toggling bit values, LSB, application of set operations. **String ADT:** The concatenation of two strings, the extraction of substrings, searching a string for a matching substring, parsing.

3. Course Materials

- **Textbook:** Introduction to algorithms by Thomas H. Cormen[et al.].—3rd ed.
- **Powerpoint slides** and other additional materials will be shared on Google Classroom.

4. Lecture Plan

Lecture	Topics
1	**Introduction:** Introduction to Data Structures, idea of abstract data type.
2	**Complexity:** preliminary idea of algorithm runtime complexity (Big O

	notation), preliminary idea of data structure space complexity.
3	**LinkedList:** Singly/doubly/circular linked lists, basic operations on linked list (insertion, deletion and traverse), dynamic array and its application.
4	**Stack:** Basic stack operations (push/pop/peek), stack-class implementation using Array and linked list, in-fix to post-fix expressions conversion and evaluation, balancing parentheses using stack.
5	**Queue:** Basic queue operations(enqueue, dequeue), circular queue/ dequeue, queue-class implementation using array and linked list, application- Josephous problem, palindrome checker using stack and queue.
6	**Sorting I:** Insertion sort, selection sort, bubble sort.
7	**Sorting II:** distribution sort (counting sort, radix sort, bucket sort), lower bounds for sorting, external sort.
8	**Searching:** Linear search, binary Search, application of Binary Search- finding element in a sorted array, finding n^{th} root of a real number, solving equations.
9	**Recursion:** Basic idea of recursion (3 laws-base case, call itself, move towards base case by state change), tracing output of a recursive function,
10	**Recursion II:** Applications- merge sort, quick sort.
11	**Recursion III:** Applications-permutation, combination.
12	**Binary Tree:** Binary tree representation using array and pointers, traversal of Binary Tree (in-order, pre-order and post-order).
13	**Binary Search Tree:** BST representation, basic operations on BST (creation, insertion, deletion, querying and traversing), application- searching, sets.
Incourse	
14	**Heap:** Min-heap, max-heap.
15	**Heap II:** Fibonacci-heap.

16	**Heap III:** applications-priority queue, heap sort.
17	**Self-balancing Binary Search Tree:** AVL tree (rotation, insertion).
18	**Self-balancing Binary Search Tree II:** Red–black tree
19	**Disjoint Set:** Union find, path compression.
20	**Huffman Coding:** Implementation, application- Compression.
21	**Graph I:** Graph representation (adjacency matrix/adjacency list), basic operations on graph (node/edge insertion and deletion).
22	**Graph II:** Traversing a graph: breadth-first search (BFS).
23	**Graph III:** Traversing a graph: depth-first search (DFS).
24	**Graph IV:** Applications of graph - graph bicoloring.
25	**Set Operations:** Set representation using bitmask, set/clear bit, querying the status of a bit, toggling bit values, LSB, application of set operations.
26	**String ADT:** The concatenation of two strings, the extraction of substrings, searching a string for a matching substring, parsing, suffix tree, trie.
27	**Advanced Tree I:** Segment Tree
28	**Advanced Tree II:** Binary Indexed Tree

5. Evaluation and Grading

This course will be evaluated out of 100 marks including continuous assessments and final examinations following the grading policy of the University of Dhaka for regular undergraduate and graduate degree programs. Below are the tentative marks distribution.

Component	**Marks**
Incourse	15
Quiz	10
Assignment	5

Final Exam	70

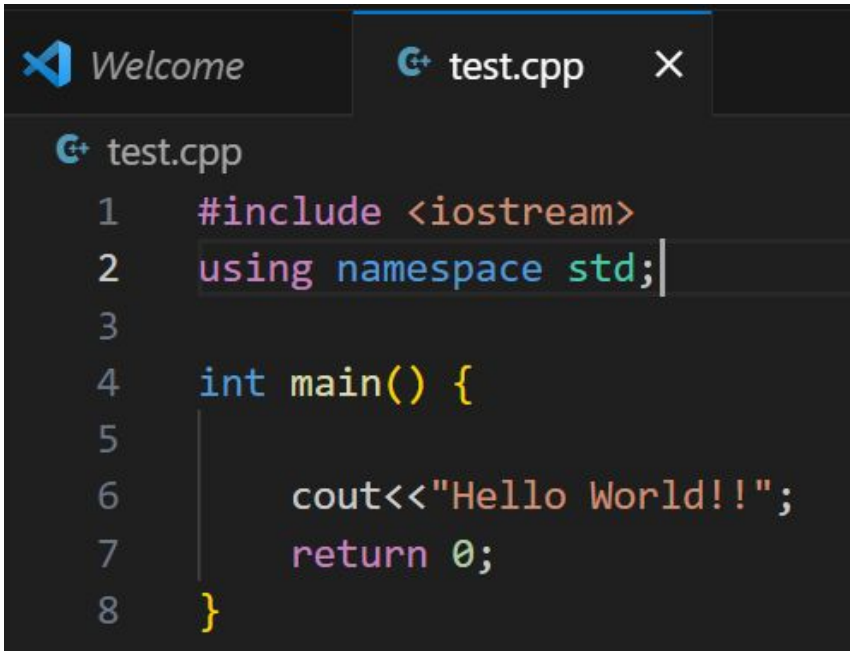
6. Academic Dishonesty

Any act of academic dishonesty including the adoption of unfair means in the examinations, copying from others, and submission of plagiarized term paper or case presentation or any designated report exercised by a student will result in an 'F' grade in the concerned course subject to the determination of the instructors.

Introduction

Md. Tanvir Alam

Introduction to C++



How to run?

```
tanvir@DESKTOP-QQQFRN5:~/DS Lab$ g++ test.cpp && ./a.out
Hello World!!
tanvir@DESKTOP-QQQFRN5:~/DS Lab$
```

g++ version

```
tanvir@DESKTOP-QQQFRN5:~/DS Lab$ g++ --version
g++ (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

tanvir@DESKTOP-QQQFRN5:~/DS Lab$
```

More Examples

Welcome | test.cpp

test.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n = 6;
    cout<<n<<endl;
    return 0;
}
```

```
tanvir@DESKTOP-QQQFRN5:~/DS Lab$ g++ test.cpp && ./a.out
6
tanvir@DESKTOP-QQQFRN5:~/DS Lab$
```

More Examples

Welcome | test.cpp

test.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin>>n;
    cout<<n+1<<endl;
    return 0;
}
```

```
tanvir@DESKTOP-QQQFRN5:~/DS Lab$ g++ test.cpp && ./a.out
3
4
tanvir@DESKTOP-QQQFRN5:~/DS Lab$
```

More Examples

Welcome | test.cpp

test.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n, m;
    cin>>n>>m;
    cout<<n+1<<" "<<m+2<<endl;
    return 0;
}
```

```
tanvir@DESKTOP-QQQFRN5:~/DS Lab$ g++ test.cpp && ./a.out
2
3
3 5
tanvir@DESKTOP-QQQFRN5:~/DS Lab$
```

Problem

There are three points in a 3d space.

The position of a point is presented with its three coordinate values (x, y, z).

Take the positions as input.

Print the positions as output.

Naive Solution

Welcome | test.cpp

test.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int p1x, p1y, p1z, p2x, p2y, p2z,p3x, p3y, p3z;
    cin>>p1x>>p1y>>p1z>>p2x>>p2y>>p2z>>p3x>>p3y>>p3z;
    cout<<p1x<<p1y<<p1z<<p2x<<p2y<<p2z<<p3x<<p3y<<p3z;
    return 0;
}
```

What if there are more attributes of a point? For example, color, size…

Struct

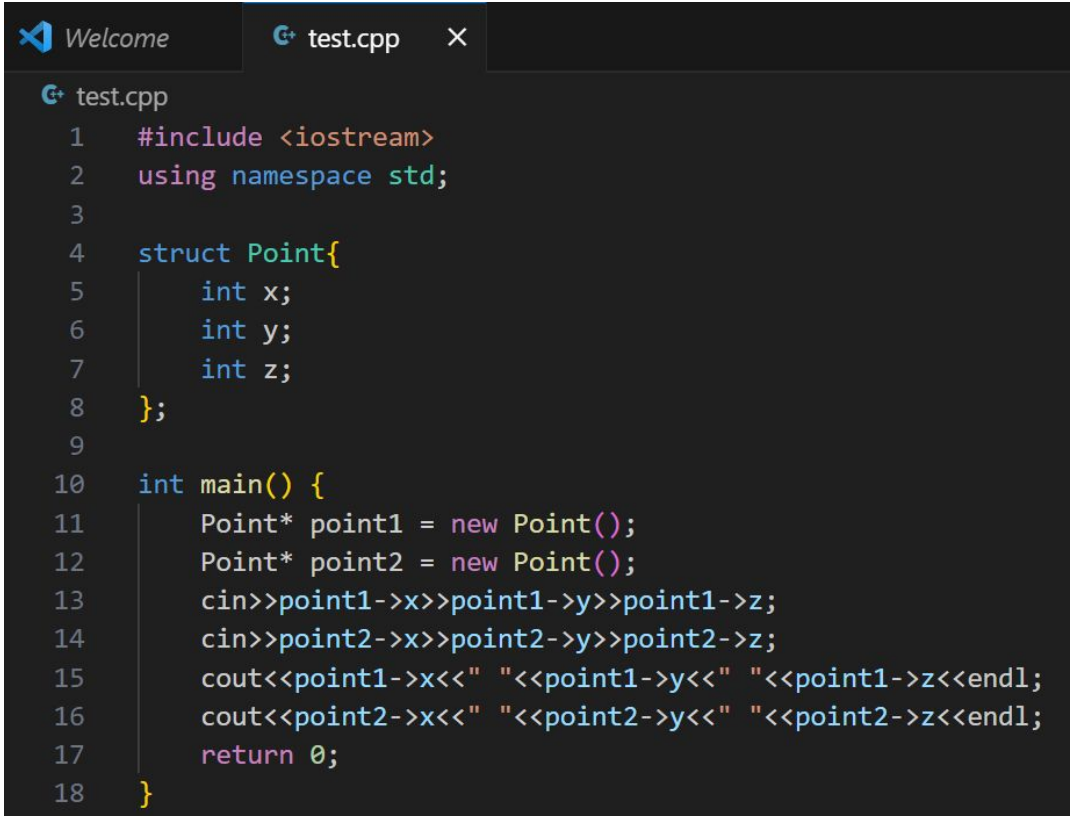
Welcome | test.cpp

test.cpp

```cpp
#include <iostream>
using namespace std;

struct Point{
    int x;
    int y;
    int z;
};

int main() {
    Point* p = new Point();
    p->x = 2;
    p->y = 3;
    p->z = 4;
    cout<<p->x<<endl;
    cout<<p->y<<endl;
    cout<<p->z<<endl;
    return 0;
}
```

```
tanvir@DESKTOP-QQQFRN5:~/DS Lab$ g++ test.cpp && ./a.out
2
3
4
```

Better Solution



Another Example

Welcome | test.cpp

test.cpp

```cpp
#include <iostream>
using namespace std;

struct Student{
    string name = "Bob";
    int marks = 20;
};

int main() {
    Student* st = new Student();
    cout<<st->name<<endl;
    cout<<st->marks<<endl;
    return 0;
}
```

```
tanvir@DESKTOP-QQQFRN5:~/DS Lab$ g++ test.cpp && ./a.out
Bob
20
```

Function in struct

Welcome | test.cpp

test.cpp

```cpp
#include <iostream>
using namespace std;

struct Student{
    string name = "Bob";
    int marks = 20;

    void increaseMarks(int increment){
        marks += increment;
    }
};

int main() {
    Student* st = new Student();
    cout<<st->name<<endl;
    cout<<st->marks<<endl;
    st->increaseMarks(5);
    cout<<st->marks<<endl;
    return 0;
}
```

```
tanvir@DESKTOP-QQQFRN5:~/DS Lab$ g++ test.cpp && ./a.out
Bob
20
25
```

Constructor in struct

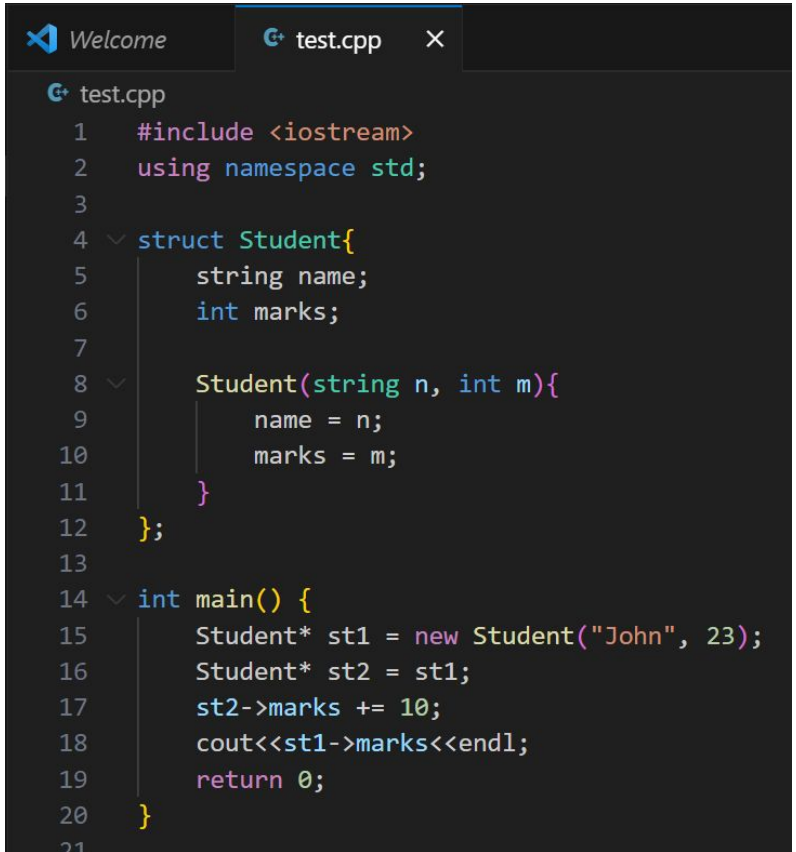
Welcome | test.cpp

test.cpp

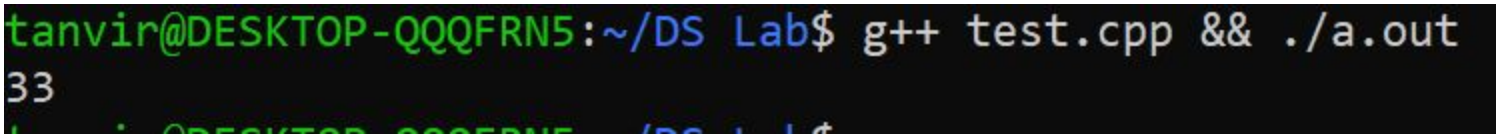
```cpp
#include <iostream>
using namespace std;

struct Student{
    string name;
    int marks;

    Student(string n, int m){
        name = n;
        marks = m;
    }
};

int main() {
    Student* st = new Student("John", 23);
    cout<<st->name<<endl;
    cout<<st->marks<<endl;
    return 0;
}
```

```
tanvir@DESKTOP-QQQFRN5:~/DS Lab$ g++ test.cpp && ./a.out
John
23
```

Determine the output



Answer



Welcome | test.cpp

test.cpp

```cpp
#include <iostream>
using namespace std;

struct Student{
    string name;
    int marks;

    Student(string n, int m){
        name = n;
        marks = m;
    }
};

int main() {
    Student* st1 = new Student("John", 23);
    Student* st2 = st1;
    Student* st3 = new Student("Bob", 30);
    cout<<st1<<endl;
    cout<<st2<<endl;
    cout<<st3<<endl;
    return 0;
}
```

```
tanvir@DESKTOP-QQQFRN5:~/DS Lab$ g++ test.cpp && ./a.out
0x5569c7cc5eb0
0x5569c7cc5eb0
0x5569c7cc5ee0
```

Thank you

Complexity

Md. Tanvir Alam

Good Algorithm vs Bad Algorithm

- Computation time
- Memory or Space

Big “O”

```
Def Sum(inp_arr):
    Sum = 0              ---------------------- b
    For i in inp_arr:
        Sum += i         ---------------------- a
    Return Sum           ---------------------- c
```

Big “O”

```
Def Sum(inp_arr):
    Sum = 0
    For i in inp_arr:
        Sum += i
    Return Sum
```

$T(n) = an + b + c = O(n)$

Linear time

Big “O”

```
Def FirstItemPlusOne(inp_arr):
        F = inp_arr[0] +1      ---------------------- c
        Return F               ---------------------- b
```

T(n) = b + c = O(1)

Big “O”

```
Def FindDuplicate(inp_arr):
    For i in (0,n-1):                    ---------- n times
        For j in (0,n-1):                ---------- n times
            If i!=j and inp_arr[i] == inp_arr[j]:
                Return True
    Return False
```

$T(n) = n \times n \times c = O(n^2)$

Big “O”

```
Def FindDuplicateAndSum(inp_arr):
        Dup, Sum = False, 0
        For i in (0,n-1):                          ------------------------- n times
                For j in (0,n-1):                  ------------------------- n times
                        If i!=j and inp_arr[i] == inp_arr[j]:
                                Dup = True
        For i in (0,n-1):
                Sum += inp_arr[i]
        Return Dup, Sum
```

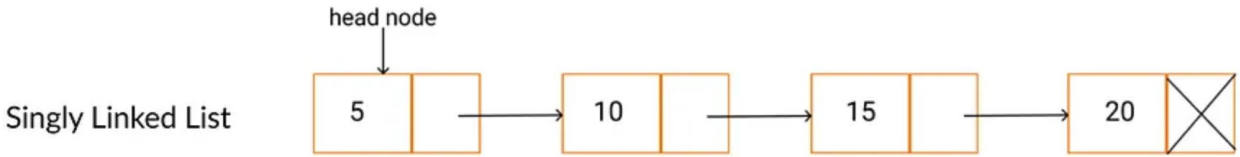
$T(n) = n \times n \times c + bn = O(n^2)$

Thank You

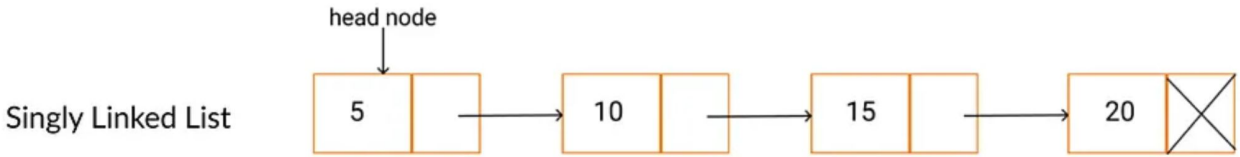
Linked List

Md. Tanvir Alam

Linked List?

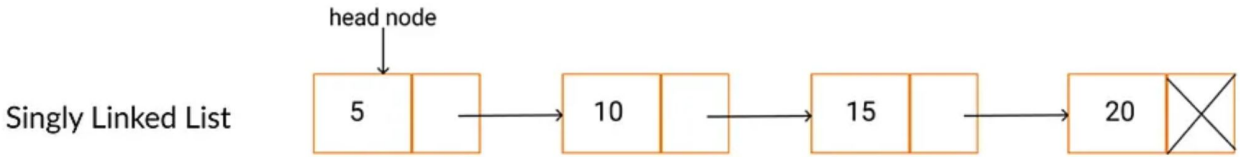


Linked List?




```
struct node{



};
```

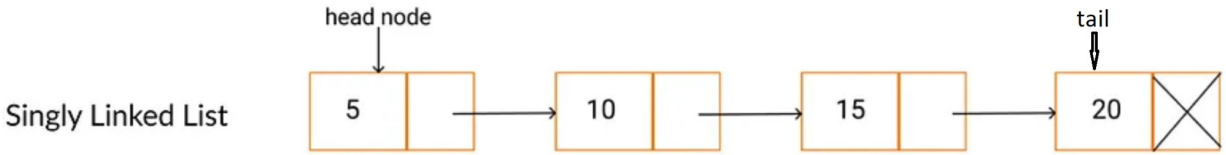
Linked List?




```
struct node{
        node *next;
        int val;
};
```

Linked List Structure

```
struct ll{
    struct node{
        node *next;
        int val;
    };
    node *head=NULL;
    node *tail=NULL;
};
```



Insert at First

```
void insert_first(int x){
        //insert x at first
        node *a = (node*)malloc(sizeof(node));
        a->next=NULL;
        a->val=x;
        if(head==NULL){
            head=a;
            tail=a;
        }
        else{
            a->next=head;
            head=a;
        }
    }
```

Insert at Last

```
void insert_last(int x){
        //insert x at last
        node*a=(node*)malloc(sizeof(node));
        a->next=NULL;
        a->val=x;
        if(tail){
            tail->next=a;
            tail=a;
        }
        else{
            head=a;
            tail=a;
        }
    }
```

Traverse

```
void print(){
        node *p=head;
        while(p)
        {
            printf("%d ",p->val);
            p=p->next;
        }
}
```

Delete First Element

```
int delete_f()
    {
        //delete first element
        if(head==NULL)
            return -1;
        if(head==tail)
        {
            int x=head->val;
            head=NULL;
            tail=NULL;
            return x;
        }
        else
        {
            int x=head->val;
            head=head->next;
            return x;
        }
    }
```

Delete Last Element

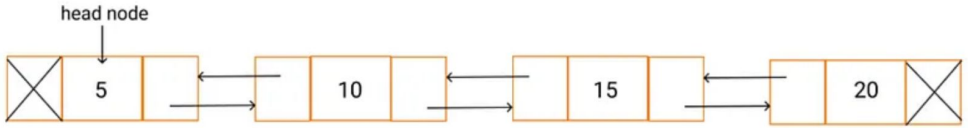
Delete

```
void delfh(int x){
        //delete the first x
        node *p=head;
        node *tmp;
        if(head->val==x){
            head = head->next;
        }
        while(p->next)
        {
            if(p->next->val==x)
            {
                p->next = p->next->next;
                break;
            }
            p=p->next;
        }
}
```

Doubly Linked List

```
struct dll
{
    struct node
    {
        node* prev;
        node* next;
        int val;
    };
    node* head=NULL;
    node* tail=NULL;
};
```

Doubly Linked List



Insert at First

```
void insert_first(int x)
   {
      node* a=(node*)malloc(sizeof(node));
      a->prev=NULL;
      a->next=NULL;
      a->val=x;
      if(head==NULL)
      {
         head=a;
         tail=a;
      }
      else
      {
         a->next=head;
         head->prev=a;
         head=head->prev;
      }
   }
```

Insert at Last

```
void insert_last(int x){
        node* a=(node*)malloc(sizeof(node));
        a->prev=NULL;
        a->next=NULL;
        a->val=x;
        if(head==NULL)
        {
            head=a;
            tail=a;
        }
        else
        {
            a->prev=tail;
            tail->next=a;
            tail=tail->next;
        }
    }
```

Traverse

```
void print()
    {
        node* p=head;
        while(p)
        {
            printf("%d ",p->val);
            p=p->next;
        }
    }
```

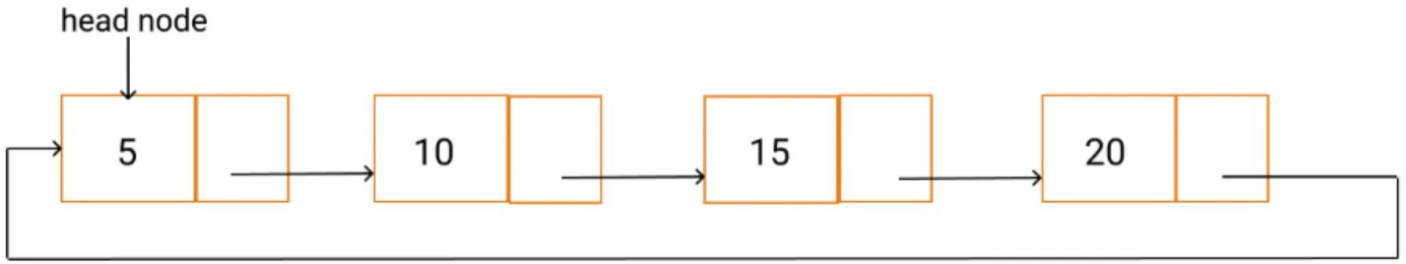
Delete First Element

```
int delete_first()
   {
      if(head==NULL)
         return -1;
      else if(head->next==NULL)
      {
         int x=head->val;
         head=NULL;
         tail=NULL;
         return x;
      }
      else
      {
         int x=head->val;
         head=head->next;
         head->prev=NULL;
         return x;
      }
   }
```

Delete Last Element

```
int delete_last()
   {
      if(head==NULL)
         return -1;
      else if(head->next==NULL)
      {
         int x=head->val;
         head=NULL;
         tail=NULL;
         return x;
      }
      else
      {
         int x=tail->val;
         tail=tail->prev;
         tail->next=NULL;
         return x;
      }
   }
```

Circular Linked List



Why Linked List?

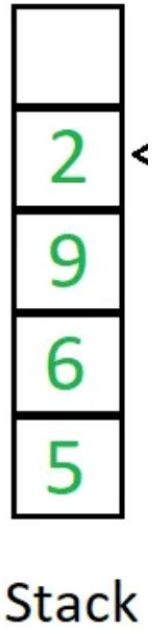
- Can grow and shrink at runtime
- No memory wastage

Stack

Md. Tanvir Alam

Basic Stack Operations

- Push
- Pop
- Peek



Stack

Implementing Stack using Array

Implementing Stack using Linked List

Why Linked List instead of Array?

Infix to Postfix

Infix expression

operator is in-between operands

(a + b)

A + B * C + D

A + B * (C + D)

Postfix expression

operator is in-between operands

(ab+)

ABC*+D+

ABCD+*+

Infix Expression to a Postfix Expression

- Scan input string from left to right character by character.
- If the character is an operand, put it into output stack.
- If the character is an operator and operator's stack is empty, push operator into operators' stack.
- If the operator's stack is not empty, there may be following possibilities.
- If the precedence of scanned operator is greater than the top most operator of operator's stack, push this operator into operator's stack.

Infix Expression to a Postfix Expression

- If the precedence of scanned operator is less than or equal to the top most operator of operator's stack, pop the operators from operator’s stack until we find a low precedence operator than the scanned character. Never pop out ('(') or (')') whatever may be the precedence level of scanned character.
- If the character is opening round bracket ('('), push it into operator's stack.
- If the character is closing round bracket (')'), pop out operators from operator's stack until we find an opening bracket ('(').
- Now pop out all the remaining operators from the operator's stack and push into output stack.

Postfix Evaluation

Infix Expression to a Postfix Expression

- Iterate the expression from left to right
- Keep on storing the operands into a stack.
- Once an operator is received, pop the two topmost elements
- Evaluate them
- Push the result in the stack again.

Problem A

In this problem, you will have a linked list of sorted linked lists containing integers. You will have to merge the linked lists of integers into a single sorted linked list of integers.

You must use the given template.

Input:
First line: n, the number of linked lists. (1<=n<=100)
For 1<=i<=n:

Next line: m_i, an integer (1<=v <=10000), the number of integers in i-th linked list.
Next m_i lines: v_j, an integer (1<=j<=m_i, -2147483648<=v_j <=2147483647), the j-th value of the i-th linked list.

Output:
Each line contains the values in the sorted linked list.

Sample Case:

Input	Output
3 3 3 5 9 2 4 7 2 6 8	3 4 5 6 7 8 9

Problem B

In this problem, you will have a ternary tree of integers where each node has up to three children: left child, mid-child, and right child. You will have to print the tree in a new order that prints the tree in the following order: left sub-tree, mid sub-tree, node value, and finally the right sub-tree.

Input:
First line: r, the value of root. (-2147483648<=r<=2147483647)
Next line: n, the number of operations. (1<=n<=10000)
Next n lines: *op key val*, three integers (0<=*op*<=2, -2147483648<=*key, val*<=2147483647). If op = 0, set the left child of the node with the value *key* to *val*. If op = 1, set the mid child of the node with the value *key* to *val*. If op = 2, set the right child of the node with the value *key* to *val*. If the key is not found, ignore the command.

Output:
Each line contains the values in the tree according to the new order.

Sample Case:

Input	Output
5 5 0 5 6 1 5 7 2 5 8 0 7 9 2 7 10	6 9 7 10 5 8

Problem C

In this problem, you will have to check whether two binary trees are equal or not.

Input:
First line: r1, the value of root of the first tree. (-2147483648<=r1<=2147483647)
Next line: n1, the number of operations. (1<=n1<=10000)
Next n lines: *op key val*, three integers (0<=*op*<=1, -2147483648 <= *key, val* <= 2147483647). If op = 0, set the left child of the node in the first tree with the value *key* to *val*. If op = 1, set the right child of the node in the first tree with the value *key* to *val*. If the key is not found, ignore the command.
Next line: r2, the value of root of the second tree. (-2147483648<=r2<=2147483647)
Next line: n2, the number of operations. (1<=n2<=10000)
Next n lines: *op key val*, three integers (0<=*op*<=1, -2147483648 <= *key, val* <= 2147483647). If op = 0, set the left child of the node in the second tree with the value *key* to *val*. If op = 1, set the right child of the node in the second tree with the value *key* to *val*. If the key is not found, ignore the command.

Output:
1, if the trees are equal. 0, otherwise.

Sample Case:

Input	Output
5 4 0 5 6 1 5 7 0 7 9 1 7 10 5 4 0 5 6 1 5 9 1 9 10 0 10 7	0
5 4 0 5 6 1 5 7	1

0 7 9 1 7 10 5 4 0 5 6 1 5 7 1 7 10 0 7 9	

Problem D

In this problem, you will have to implement a binary search tree.

You must use the given template.

Input:
First line: n, an integer. (1<=n<=10000)
Next n lines: v (-2147483648<=v<=2147483647), an integer to be inserted in the binary search tree.
Next line: m, an integer. (1<=m<=10000)
Next m lines: k (-2147483648<=k<=2147483647), an integer to be deleted from the binary search tree.

Output:
Each line contains the values in the tree according to the in-order traversal.

Sample Case:

Input	Output
10 6 5 7 8 4 3 9 0 1 2 3 2 4 6	0 1 3 5 7 8 9